SVM_RecursosHumanos

October 28, 2026

[198]:
```
import pandas as pd
import pylab as pl
import numpy as np
import scipy.optimize as opt
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings("ignore")
```

[199]:
```
# Carga de la base de datos
import os

cell_df = pd.read_csv("recursos_humanos (3).csv")
cell_df
```

[199]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	left
0	0.38	0.53	2	157	3	0	1
1	0.80	0.86	5	262	6	0	1
2	0.11	0.88	7	272	4	0	1
3	0.72	0.87	5	223	5	0	1
4	0.37	0.52	2	159	3	0	1
…	…	…	…				
14994	0.40	0.57	2				
14995	0.37	0.48	2				
14996	0.37	0.53	2				
14997	0.11	0.96	6				
14998	0.37	0.52	2				

```
…                       …                   …              …   …
14994                   151                 3              0   1
14995                   160                 3              0   1
14996                   143                 3              0   1
14997                   280                 4              0   1
14998                   158                 3              0   1

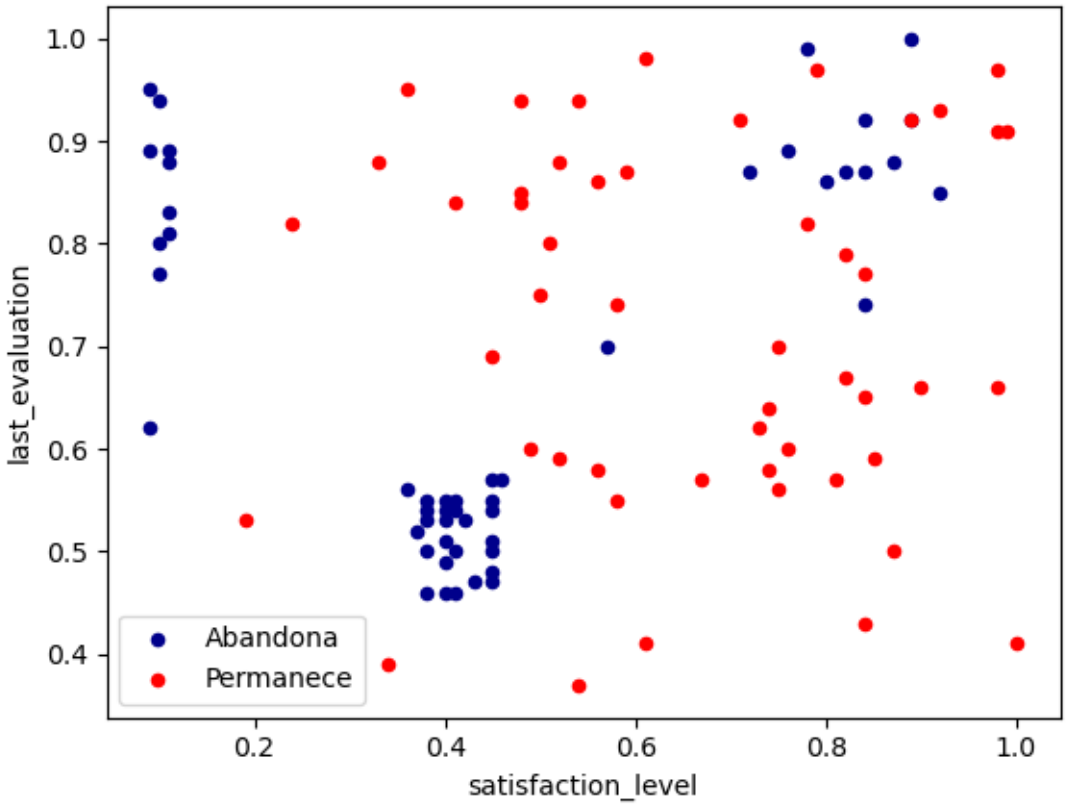
       promotion_last_5years    sales  salary
0                          0    sales     low
1                          0    sales  medium
2                          0    sales  medium
3                          0    sales     low
4                          0    sales     low
…                        …      …      …
14994                      0  support     low
14995                      0  support     low
14996                      0  support     low
14997                      0  support     low
14998                      0  support     low

[14999 rows x 10 columns]
```

[200]:
```
# Identificación de las clases
cell_df["left"].value_counts()
```

[200]:
```
left
0    11428
1     3571
Name: count, dtype: int64
```

[201]:
```
# Gráfica de empleados que permanecen y abandonan la empresa
ax = cell_df[cell_df["left"] == 1][0:50].
 ↪plot(kind="scatter",x="satisfaction_level",y="last_evaluation",color="DarkBlue",label="Aban
ax = cell_df[cell_df["left"] == 0][0:50].
 ↪plot(kind="scatter",x="satisfaction_level",y="last_evaluation",color="Red",label="Permanece
plt.show()
```



[202]:
```
# Preprocesamiento de datos y selección
cell_df.dtypes
```

[202]:
```
satisfaction_level       float64
last_evaluation          float64
number_project             int64
average_montly_hours       int64
time_spend_company         int64
Work_accident              int64
left                       int64
promotion_last_5years      int64
sales                     object
salary                    object
dtype: object
```

[165]:
```
# Revisión de valores faltantes
cell_df.isnull().sum()
```

[165]:
```
satisfaction_level       0
last_evaluation          0
number_project           0
```

```
average_montly_hours     0
time_spend_company       0
Work_accident            0
left                     0
promotion_last_5years    0
sales                    0
salary                   0
dtype: int64
```

[166]:
```
# Conversión de variables categóricas
cell_df = pd.get_dummies(cell_df,columns=["sales", "salary"],dtype=int)
cell_df
```

[166]:

	satisfaction_level	last_evaluation	number_project	…
0	0.38	0.53	2	
1	0.80	0.86	5	
2	0.11	0.88	7	
3	0.72	0.87	5	
4	0.37	0.52	2	
…	…	…	…	
14994	0.40	0.57	2	
14995	0.37	0.48	2	
14996	0.37	0.53	2	
14997	0.11	0.96	6	
14998	0.37	0.52	2	

	average_montly_hours	time_spend_company	Work_accident	left	…
0	157	3	0	1	
1	262	6	0	1	
2	272	4	0	1	
3	223	5	0	1	
4	159	3	0	1	
…	…	…	…	…	
14994	151	3	0	1	
14995	160	3	0	1	
14996	143	3	0	1	
14997	280	4	0	1	
14998	158	3	0	1	

	promotion_last_5years	sales_IT	sales_RandD	…	sales_hr
0	0	0	0	…	0
1	0	0	0	…	0
2	0	0	0	…	0
3	0	0	0	…	0
4	0	0	0	…	0
…	…	…	…	…	…
14994	0	0	0	…	0

```
14995                    0         0            0  …         0
14996                    0         0            0  …         0
14997                    0         0            0  …         0
14998                    0         0            0  …         0

       sales_management  sales_marketing  sales_product_mng  sales_sales  \
0                     0                0                  0            1
1                     0                0                  0            1
2                     0                0                  0            1
3                     0                0                  0            1
4                     0                0                  0            1
…                   …                …                  …          …
14994                 0                0                  0            0
14995                 0                0                  0            0
14996                 0                0                  0            0
14997                 0                0                  0            0
14998                 0                0                  0            0

       sales_support  sales_technical  salary_high  salary_low  salary_medium
0                  0                0            0           1              0
1                  0                0            0           0              1
2                  0                0            0           0              1
3                  0                0            0           1              0
4                  0                0            0           1              0
…                …                …          …         …            …
14994              1                0            0           1              0
14995              1                0            0           1              0
14996              1                0            0           1              0
14997              1                0            0           1              0
14998              1                0            0           1              0

[14999 rows x 21 columns]
```

0.0.1 1. Preparación de la base de datos

Se cargó la base de datos de recursos humanos, que contiene información de 14,999 empleados. Primero revisamos los tipos de variables y comprobamos que no existieran valores faltantes.

Después, utilizamos `pd.get_dummies()` para convertir las variables categóricas `sales` y `salary` en variables numéricas, ya que el modelo SVM necesita trabajar con números.

La variable que buscamos predecir es `left`, donde 0 significa que el empleado permanece en la empresa y 1 significa que la abandona.

Finalmente, separamos las variables independientes de la variable dependiente para comenzar con el entrenamiento del modelo.

[167]:
```
# Verificación de tipos de datos
cell_df.dtypes
```

[167]:
```
satisfaction_level       float64
last_evaluation          float64
number_project             int64
average_montly_hours       int64
time_spend_company         int64
Work_accident              int64
left                       int64
promotion_last_5years      int64
sales_IT                   int64
sales_RandD                int64
sales_accounting           int64
sales_hr                   int64
sales_management           int64
sales_marketing            int64
sales_product_mng          int64
sales_sales                int64
sales_support              int64
sales_technical            int64
salary_high                int64
salary_low                 int64
salary_medium              int64
dtype: object
```

[168]:
```
# Selección de variables independientes
feature_df = cell_df.drop(columns=["left"])
x = np.asarray(feature_df)
x[0:5]
```

[168]:
```
array([[3.80e-01, 5.30e-01, 2.00e+00, 1.57e+02, 3.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        1.00e+00, 0.00e+00],
       [8.00e-01, 8.60e-01, 5.00e+00, 2.62e+02, 6.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 1.00e+00],
       [1.10e-01, 8.80e-01, 7.00e+00, 2.72e+02, 4.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 1.00e+00],
       [7.20e-01, 8.70e-01, 5.00e+00, 2.23e+02, 5.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        1.00e+00, 0.00e+00],
       [3.70e-01, 5.20e-01, 2.00e+00, 1.59e+02, 3.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
        0.00e+00, 0.00e+00, 1.00e+00, 0.00e+00, 0.00e+00, 0.00e+00,
```

```
       1.00e+00, 0.00e+00]])
```

[169]:
```
# Selección de la variable dependiente
cell_df["left"] = cell_df["left"].astype("int")
y = np.asarray(cell_df["left"])
y[0:5]
```

[169]:
```
array([1, 1, 1, 1, 1])
```

[170]:
```
# Creación de grupos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2,␣
 ↪random_state=4, stratify=y)
print("Grupo de entrenamiento", X_train.shape, y_train.shape)
print("Grupo de prueba", X_test.shape, y_test.shape)
```

```
Grupo de entrenamiento (11999, 20) (11999,)
Grupo de prueba (3000, 20) (3000,)
```

[171]:
```
# Estandarización de las variables
from sklearn.preprocessing import StandardScaler
escalador = StandardScaler()
X_train = escalador.fit_transform(X_train)
X_test = escalador.transform(X_test)
```

1 Modelación con SVM

1.1 Kernel RBF

[172]:
```
# Modelación con SVM - Kernel RBF
from sklearn import svm

clf = svm.SVC(kernel="rbf")
clf.fit(X_train, y_train)

yhat = clf.predict(X_test)
yhat[0:5]
```

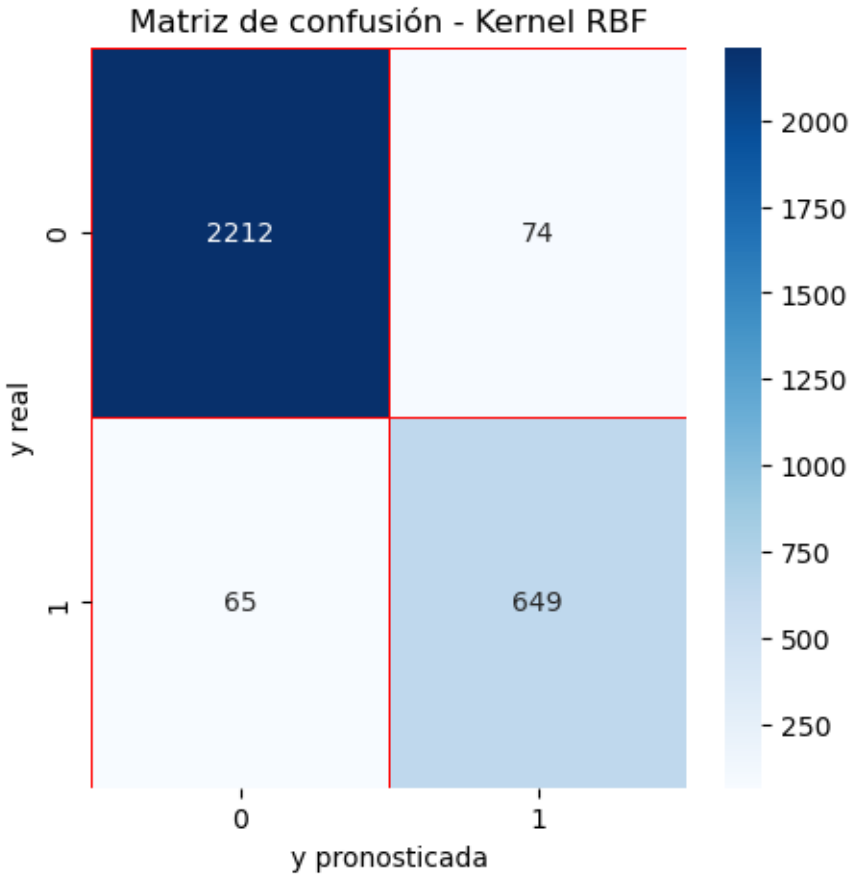
[172]:
```
array([0, 1, 0, 0, 1])
```

[173]:
```
# Creación de la matriz de confusión
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, yhat)
cm
```

[173]:
```
array([[2212,   74],
       [  65,  649]])
```

[174]:
```
# Visualización de la matriz de confusión
f, ax = plt.subplots(figsize=(5,5))

sns.heatmap(cm,annot=True,linewidths=0.5,linecolor="red",fmt=".
 ↪0f",cmap="Blues",ax=ax)
plt.xlabel("y pronosticada")
plt.ylabel("y real")
plt.title("Matriz de confusión - Kernel RBF")
plt.show()
```



1.1.1 Estadísticas de desemeño

[175]:
```
from sklearn.metrics import classification_report
print(classification_report(y_test, yhat))
```

```
              precision    recall  f1-score   support

           0       0.97      0.97      0.97      2286
           1       0.90      0.91      0.90       714
```

```
    accuracy                           0.95      3000
   macro avg       0.93      0.94      0.94      3000
weighted avg       0.95      0.95      0.95      3000
```

1.1.2 Determinación de niveles de precisión

[176]:
```
PrecisionPermanece = cm[0,0] / (cm[0,0] + cm[1,0])
PrecisionAbandona = cm[1,1] / (cm[1,1] + cm[0,1])

print("Precisión Permanece:", PrecisionPermanece)
print("Precisión Abandona:", PrecisionAbandona)
```

```
Precisión Permanece: 0.971453667105841
Precisión Abandona: 0.8976486860304288
```

1.1.3 Determinación de niveles de recuperación

[177]:
```
RecallPermanece = cm[0,0] / (cm[0,0] + cm[0,1])
RecallAbandona = cm[1,1] / (cm[1,0] + cm[1,1])

print("Recall Permanece:", RecallPermanece)
print("Recall Abandona:", RecallAbandona)
```

```
Recall Permanece: 0.9676290463692039
Recall Abandona: 0.9089635854341737
```

1.1.4 Determinación de F1score

[178]:
```
F1Permanece = 2 * (PrecisionPermanece * RecallPermanece) / (PrecisionPermanece␣
 ↪+ RecallPermanece)

F1Abandona = 2 * (PrecisionAbandona * RecallAbandona) / (PrecisionAbandona +␣
 ↪RecallAbandona)

print("F1 Permanece:", F1Permanece)
print("F1 Abandona:", F1Abandona)
```

```
F1 Permanece: 0.9695375849222004
F1 Abandona: 0.9032707028531662
```

1.1.5 Dterminación de soportes

[179]:
```
SoportePermanece = cm[0,0] + cm[0,1]
SoporteAbandona = cm[1,0] + cm[1,1]

print("Soporte Permanece:", SoportePermanece)
```

```
print("Soporte Abandona:", SoporteAbandona)
```

```
Soporte Permanece: 2286
Soporte Abandona: 714
```

1.1.6 Determinación de la precisión global

[180]:
```
Correctos = cm[0,0] + cm[1,1]
Incorrectos = cm[0,1] + cm[1,0]

PrecisionGlobal = Correctos / (Correctos + Incorrectos)

PrecisionGlobal
```

[180]:
```
0.9536666666666667
```

1.1.7 Promedios simples por indicador (Macro Average)

[181]:
```
MacroAvgPrecision = (PrecisionPermanece + PrecisionAbandona) / 2
MacroAvgRecall = (RecallPermanece + RecallAbandona) / 2
MacroF1Score = (F1Permanece + F1Abandona) / 2

print("Macro Precision:", MacroAvgPrecision)
print("Macro Recall:", MacroAvgRecall)
print("Macro F1:", MacroF1Score)
```

```
Macro Precision: 0.9345511765681349
Macro Recall: 0.9382963159016888
Macro F1: 0.9364041438876833
```

1.1.8 Promedios ponderados por indicador (Weighted Average)

[182]:
```
PesoPermanece = SoportePermanece / (SoportePermanece + SoporteAbandona)
PesoAbandona = SoporteAbandona / (SoportePermanece + SoporteAbandona)

WAvgPrecision = PrecisionPermanece * PesoPermanece + PrecisionAbandona *␣
 ↪PesoAbandona
WAvgRecall = RecallPermanece * PesoPermanece + RecallAbandona * PesoAbandona
WAvgF1Score = F1Permanece * PesoPermanece + F1Abandona * PesoAbandona

print("Weighted Precision:", WAvgPrecision)
print("Weighted Recall:", WAvgRecall)
print("Weighted F1:", WAvgF1Score)
```

```
Weighted Precision: 0.9538880816098929
Weighted Recall: 0.9536666666666667
Weighted F1: 0.9537660669897703
```

[183]:
```
# Guardar resultados del Kernel RBF
modelo_rbf = clf
yhat_rbf = yhat
from sklearn.metrics import accuracy_score, f1_score
resultados = []
resultados.append({ "Kernel": "RBF", "Accuracy": accuracy_score(y_test, yhat),␣
 ↪"F1 Abandona": f1_score(y_test, yhat)})
```

1.1.9 Kernel lineal

[184]:
```
clf = svm.SVC(kernel="linear")
clf.fit(X_train, y_train)

yhat = clf.predict(X_test)
yhat[0:5]
```

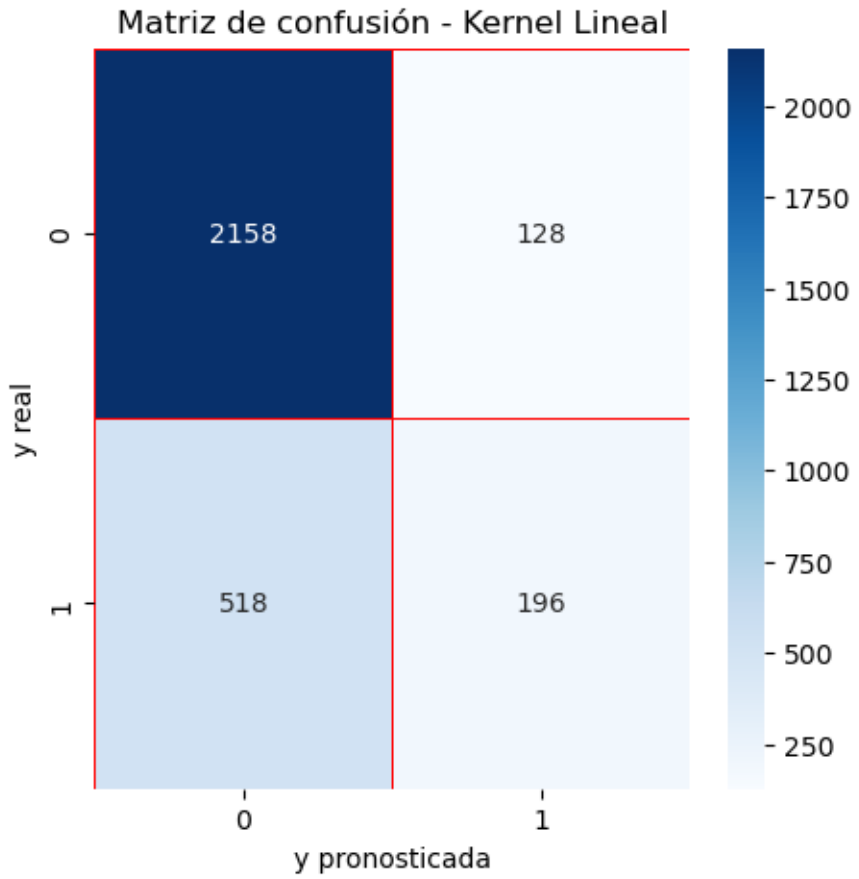
[184]:
```
array([0, 0, 0, 0, 0])
```

[185]:
```
# Creación de la matriz de confusión
cm = confusion_matrix(y_test, yhat)
cm
```

[185]:
```
array([[2158,  128],
       [ 518,  196]])
```

[187]:
```
f, ax = plt.subplots(figsize=(5,5))

sns.heatmap(cm,annot=True,linewidths=0.5,linecolor="red",fmt=".
 ↪0f",cmap="Blues",ax=ax)
plt.xlabel("y pronosticada")
plt.ylabel("y real")
plt.title("Matriz de confusión - Kernel Lineal")
plt.show()
```



[49]:
```
# Estadísticas de desempeño
print(classification_report(y_test, yhat))
```

```
              precision    recall  f1-score   support

           0       0.81      0.94      0.87      2286
           1       0.60      0.27      0.38       714

    accuracy                           0.78      3000
   macro avg       0.71      0.61      0.62      3000
weighted avg       0.76      0.78      0.75      3000
```

[188]:
```
# Guardar resultados del Kernel Lineal
modelo_lineal = clf
yhat_lineal = yhat

resultados.append({ "Kernel": "Lineal","Accuracy": accuracy_score(y_test,␣
 ↪yhat),"F1 Abandona": f1_score(y_test, yhat)})
```

1.1.10 Kernel Polinomial

[189]:
```
clf = svm.SVC(kernel="poly")
clf.fit(X_train, y_train)

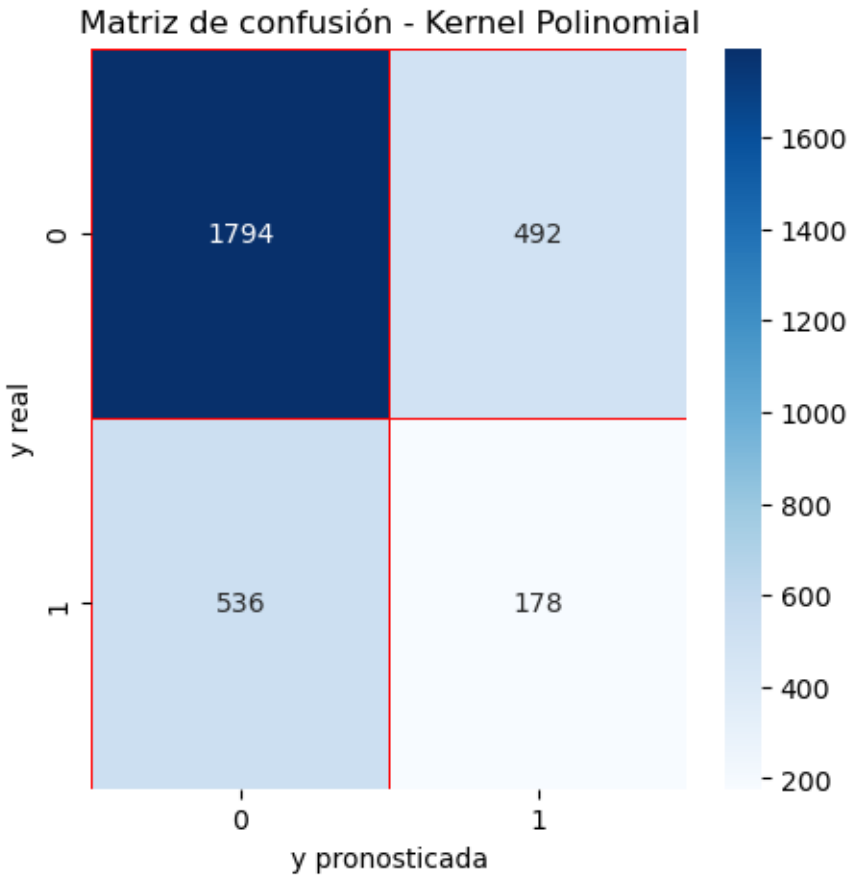
yhat = clf.predict(X_test)
yhat[0:5]

cm = confusion_matrix(y_test, yhat)
cm
```

[189]:
```
array([[2220,   66],
       [ 101,  613]])
```

[83]:
```
f, ax = plt.subplots(figsize=(5,5))

sns.heatmap(cm,annot=True,linewidths=0.5,linecolor="red",fmt=".
 ↪0f",cmap="Blues", ax=ax)
plt.xlabel("y pronosticada")
plt.ylabel("y real")
plt.title("Matriz de confusión - Kernel Polinomial")
plt.show()
```



[57]:
```
print(classification_report(y_test, yhat))
```

```
              precision    recall  f1-score   support

           0       0.96      0.97      0.96      2286
           1       0.90      0.86      0.88       714

    accuracy                           0.94      3000
   macro avg       0.93      0.91      0.92      3000
weighted avg       0.94      0.94      0.94      3000
```

[85]:
```
# Guardar resultados del Kernel Polinomial
modelo_polinomial = clf
yhat_polinomial = yhat

resultados.append({"Kernel": "Polinomial","Accuracy": accuracy_score(y_test,␣
 ↪yhat), "F1 Abandona": f1_score(y_test, yhat)})
```

1.1.11 Kernel Sigmoide

[60]:
```
clf = svm.SVC(kernel="sigmoid")
clf.fit(X_train, y_train)

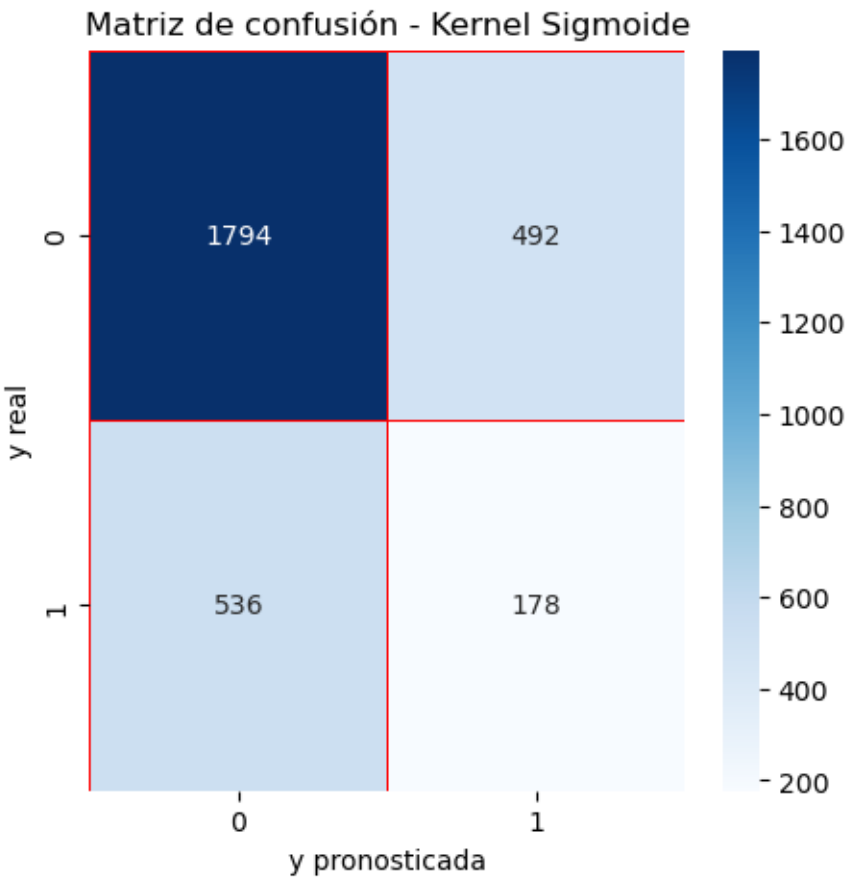
yhat = clf.predict(X_test)
yhat[0:5]

cm = confusion_matrix(y_test, yhat)
cm
```

[60]:
```
array([[1794,  492],
       [ 536,  178]])
```

[61]:
```
# Visualización de la matriz de confusión
f, ax = plt.subplots(figsize=(5,5))

sns.heatmap(cm,annot=True,linewidths=0.5,linecolor="red",fmt=".
 ↪0f",cmap="Blues",ax=ax)
plt.xlabel("y pronosticada")
plt.ylabel("y real")
plt.title("Matriz de confusión - Kernel Sigmoide")
plt.show()
```



[62]:
```
print(classification_report(y_test, yhat))
```

```
              precision    recall  f1-score   support

           0       0.77      0.78      0.78      2286
           1       0.27      0.25      0.26       714

    accuracy                           0.66      3000
   macro avg       0.52      0.52      0.52      3000
weighted avg       0.65      0.66      0.65      3000
```

[87]:
```
# Guardar resultados del Kernel Sigmoide
modelo_sigmoide = clf
yhat_sigmoide = yhat

resultados.append({"Kernel": "Sigmoide","Accuracy": accuracy_score(y_test,␣
 ↪yhat),"F1 Abandona": f1_score(y_test, yhat)})
```

1.1.12 Tabla comparativa

[67]:
```
comparacion = pd.DataFrame(resultados)
comparacion = comparacion.sort_values(by="F1 Abandona",ascending=False)
comparacion
```

[67]:

	Kernel	Accuracy	F1 Abandona
0	RBF	0.953667	0.903271
2	Polinomial	0.944333	0.880115
3	Polinomial	0.944333	0.880115
1	Lineal	0.784667	0.377649
4	Sigmoide	0.657333	0.257225

1.1.13 2. Aplicación e interpretación de los modelos SVM

Para realizar la clasificación de los empleados utilizamos Máquinas de Soporte Vectorial (SVM), probando cuatro tipos de Kernel: RBF, Lineal, Polinomial y Sigmoide.

La base de datos se dividió en 80% para entrenamiento y 20% para prueba. Después, entrenamos cada modelo y obtuvimos su matriz de confusión y reporte de clasificación.

Interpretación de los resultados:

- **RBF:** Obtuvo los mejores resultados, con una exactitud global del 95.37%. Su F1-Score de 90.33% indica un buen equilibrio entre la precisión y la recuperación de los empleados que abandonan la empresa.
- **Lineal:** Alcanzó una exactitud del 78.47%, pero su F1-Score para los empleados que abandonan fue de 37.76%, por lo que tiene dificultades para identificar correctamente esta clase.
- **Polinomial:** Presentó un buen desempeño, con una exactitud del 94.43% y un F1-Score de 88.01%. Sus resultados fueron cercanos a los del RBF.
- **Sigmoide:** Fue el modelo con el desempeño más bajo, ya que obtuvo una exactitud del 65.73% y un F1-Score de 25.72% para los empleados que abandonan la empresa.

En las matrices de confusión podemos identificar cuántos empleados fueron clasificados correctamente y cuántos fueron clasificados de manera incorrecta. Observamos que los kernels RBF y Polinomial tuvieron mejores resultados que los kernels Lineal y Sigmoide.]

[68]:
```
mejor_kernel = comparacion.iloc[0]["Kernel"]
print("El modelo más adecuado es:", mejor_kernel)
```

```
El modelo más adecuado es: RBF
```

1.1.14 3. ¿Cuál resultó ser el modelo predictivo más adecuado?

El modelo que consideramos más adecuado fue el **SVM con Kernel RBF**, porque obtuvo los mejores resultados al compararlo con los otros tres modelos.

Su exactitud global fue de **95.37%**, lo que significa que clasificó correctamente aproximadamente 95 de cada 100 empleados del conjunto de prueba.

Además, obtuvo una precisión de 89.76%, un Recall de 90.90% y un F1-Score de 90.33% para los empleados que abandonan la empresa.

Elegimos este modelo porque no solamente tiene una buena exactitud general, sino que también identifica correctamente una gran parte de los empleados que abandonan la empresa, que es precisamente lo que buscamos analizar en esta actividad.

[190]:
```
# Datos del empleado
empleado = pd.DataFrame({
    "satisfaction_level": [0.5],
    "last_evaluation": [0.75],
    "number_project": [4],
    "average_montly_hours": [200],
    "time_spend_company": [4],
    "Work_accident": [0],
    "promotion_last_5years": [0],
    "sales": ["sales"],
    "salary": ["medium"]
})

empleado
```

[190]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	promotion_last_5years	sales	salary
0	0.5	0.75	4	200	4	0	0	sales	medium

[195]:
```
# Conversión de las variables categóricas
datos_empleado = pd.get_dummies(empleado, columns=["sales", "salary"],␣
 ↪dtype=int)
# Igualar las columnas con las utilizadas en el entrenamiento
datos_empleado = datos_empleado.reindex(columns=feature_df.columns,␣
 ↪fill_value=0)
# Estandarización del empleado
empleado_escalado = escalador.transform(datos_empleado)
datos_empleado
```

[195]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_company	Work_accident	promotion_last_5years	sales_IT	sales_RandD	sales_accounting	sales_hr	sales_management	sales_marketing	sales_product_mng	sales_sales	sales_support	sales_technical
0	0.5	0.75	4	200	4	0	0	0	0	0	0	0	0	0	1	0	0

```
0                  0            1              0                0

   salary_high  salary_low  salary_medium
0            0           0              1
```

[193]:
```
# Predicción del empleado con el mejor modelo SVM
pronostico = mejor_modelo.predict(empleado_escalado)
empleado["Pronóstico"] = pronostico
empleado["Resultado"] = empleado["Pronóstico"].map({
    0: "Permanece en la empresa",
    1: "Abandona la empresa"})

empleado[["salary", "Pronóstico", "Resultado"]]
```

[193]:
```
    salary  Pronóstico                Resultado
0  medium           0  Permanece en la empresa
```

[197]:
```
# Predicción del empleado con el mejor modelo SVM
pronostico = mejor_modelo.predict(empleado_escalado)
empleado["Pronóstico"] = pronostico
empleado["Resultado"] = empleado["Pronóstico"].map({
    0: "Permanece en la empresa",
    1: "Abandona la empresa"
})

# Mostrar todas las variables y el resultado
pd.set_option("display.max_columns", None)

display(empleado)
```

```
   satisfaction_level  last_evaluation  number_project  average_montly_hours  \
0                 0.5             0.75               4                   200

   time_spend_company  Work_accident  promotion_last_5years  sales  salary  \
0                   4              0                      0  sales  medium

   Pronóstico                Resultado
0           0  Permanece en la empresa
```

El modelo obtuvo un resultado de 0, lo que significa que se pronostica que el empleado permanecerá en la empresa.

De acuerdo con las características del empleado y los patrones identificados durante el entrenamiento, el modelo lo clasifica dentro del grupo de trabajadores que no abandonan la empresa.

[ ]: